AI INDUSTRIAL KNOWLEDGE INTELLIGENCE CHALLENGE · EXPERT KNOWLEDGE COPILOT

Industrial Knowledge Copilot

A RAG-powered assistant that turns fragmented plant documentation — P&IDs, work orders, SOPs, inspection reports, compliance filings — into one cited, mobile-ready knowledge base for field technicians and engineers.

Working prototype · Architecture · Methods · Demo

THE PROBLEM

Plant knowledge is fragmented — and it's costing safety, quality, and uptime

35%

of working hours lost searching for information or recreating documents that already exist

7–12

disconnected document systems at the average large plant — P&IDs, work orders, SOPs, inspections, email

18–22%

of unplanned downtime traced to maintenance decisions made without full equipment history

25%

of India's experienced industrial engineers and operators retire within the next decade

Sources: McKinsey Global Survey (2024); NASSCOM–EY manufacturing & energy study; BIS Research

THE SOLUTION

One copilot, every document format, answers you can verify

The Expert Knowledge Copilot ingests P&IDs, maintenance records, SOPs, inspection reports, and compliance filings into a single searchable index — and answers operational questions with the source, a confidence score, and a live link back to the original document.

01 **Heterogeneous ingestion**

PDFs, scanned forms, spreadsheets, email, and photos — not just clean text

02 **Mobile-first for field staff**

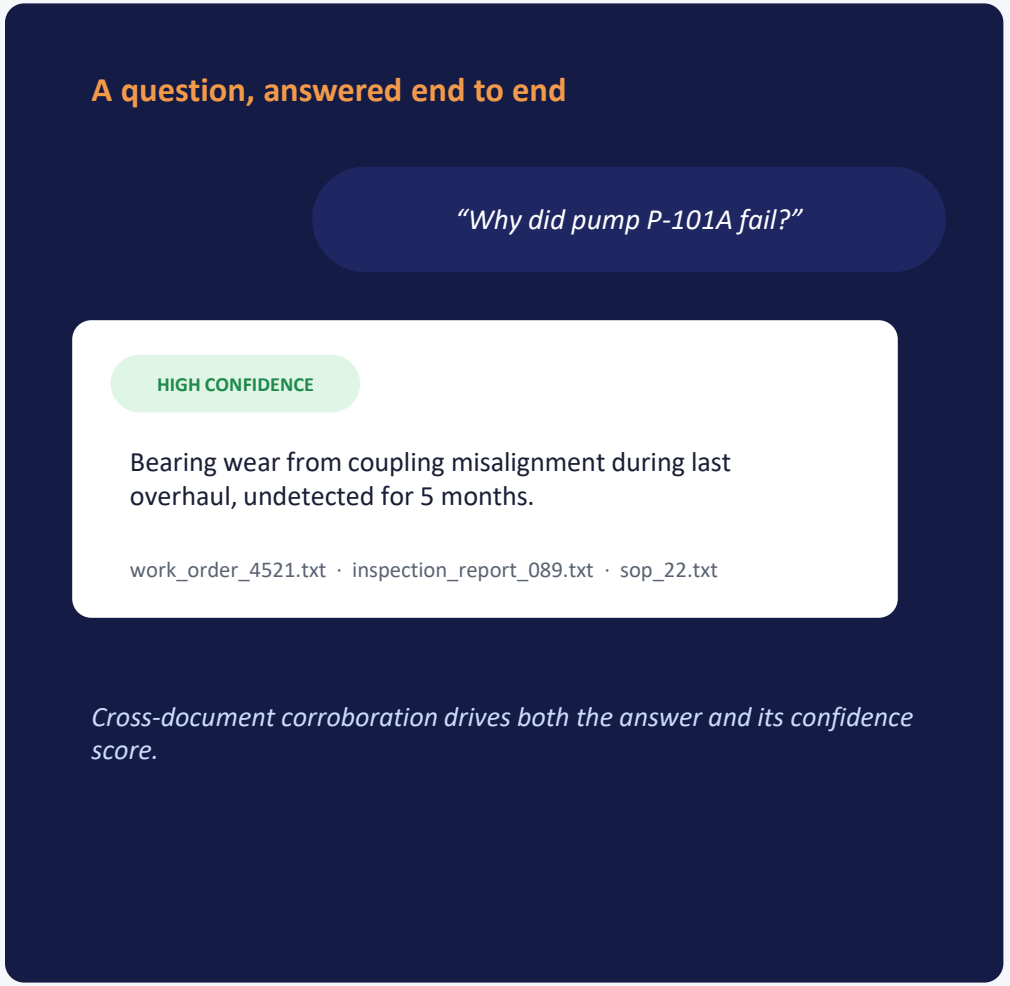
A responsive chat UI a technician can use on a phone, not just an engineer at a desk

03 **Verifiable trust signals**

Every answer carries citations, a confidence score, and a link to the source document

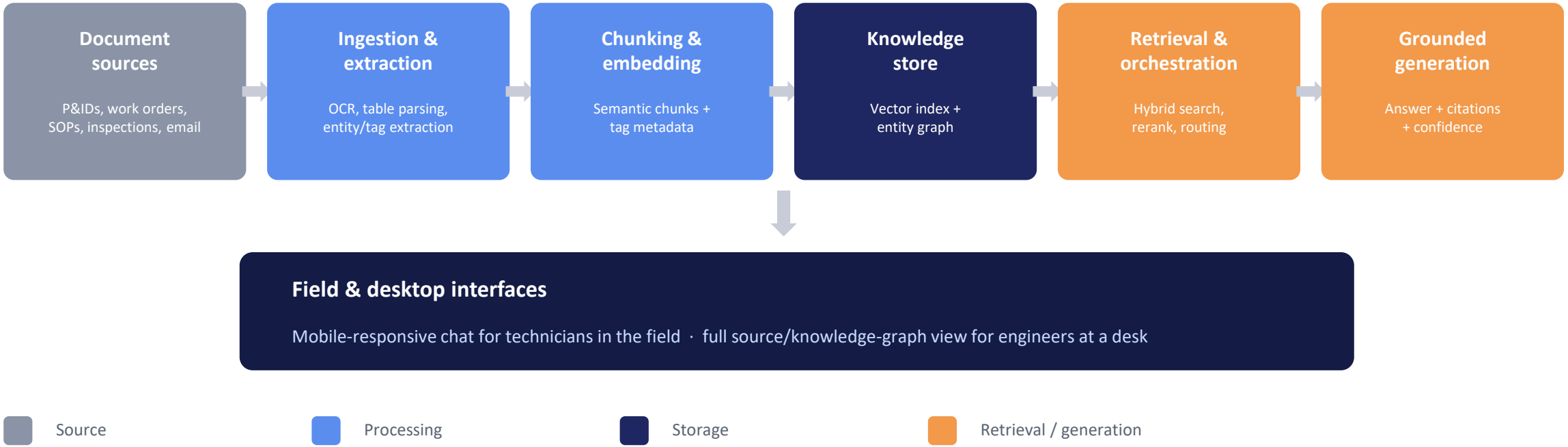
04 **Captures tacit knowledge**

Expert corrections are stored and resurfaced — a hedge against the retirement wave



ARCHITECTURE

Seven stages, one continuously updated pipeline



METHODS

What runs at each layer of the prototype

Layer	Method	Technology used
Ingestion	Format-specific extraction; OCR fallback for scans/photos	pdfplumber, python-docx, Pillow + Tesseract OCR
Chunking	Paragraph-aware splitting with sliding overlap; regex entity tagging	Custom Python (core/chunking.py)
Knowledge store	Sparse vector index of chunks; tag co-reference graph	scikit-learn TF-IDF matrix + in-memory entity graph
Retrieval	Cosine similarity search, boosted by exact equipment-tag matches	scikit-learn cosine_similarity
Confidence scoring	Weighted score from match strength + cross-document corroboration	Custom heuristic (core/index.py)
Generation	Extractive by default; grounded LLM synthesis when enabled	Rule-based extraction, or Claude via Anthropic API
Feedback loop	Expert corrections stored and resurfaced on repeat queries	JSON-backed store (core/feedback.py)
Interface	Responsive chat UI, works unmodified on a phone browser	Flask + vanilla HTML/CSS/JS

GENERATION LAYER

Which LLM is used — and why the default needs none

DEFAULT · NO LLM

Extractive answering

- TF-IDF cosine similarity retrieval over indexed chunks
- Top-matching sentences stitched into a readable answer, each tagged with its source document
- Runs fully offline — no API key, no network call, no per-query cost
- Guarantees the demo works anywhere, including with no internet at the venue

OPTIONAL · LLM-POWERED

Claude (Anthropic API)

- Enabled by setting an ANTHROPIC_API_KEY — no code change needed
- Retrieved chunks are passed as context; the model is instructed to answer only from that context and cite the source document inline
- Explicitly told to say “not enough information” rather than guess — critical for safety-related answers
- Falls back to extractive mode automatically if the API call fails

TRUST LAYER

Confidence is calculated, not decorative

Every answer scores its own reliability from two signals — so a technician knows when to double-check before acting.

```
confidence  =  0.7 × (best match score ÷ 0.5)  +  0.15 × (distinct source documents − 1)  +  0.3
```

HIGH

≥ 0.75

Strong lexical/tag match, corroborated by multiple independent documents

MEDIUM

0.50–0.75

Reasonable match, but from a single source or partial coverage

LOW

< 0.50

Weak match — surfaced with a caveat rather than presented as certain

KNOWLEDGE RETENTION

Capturing tacit knowledge before it retires

25% of India's experienced industrial engineers retire within a decade. The feedback loop below turns their corrections into permanent, reusable answers — not just document search.

1

Copilot answers

from the indexed document corpus, with a confidence score

2

Expert reviews

an engineer flags an answer as wrong or incomplete

3

Correction captured

the corrected answer is logged against that exact question

4

Resurfaced next time

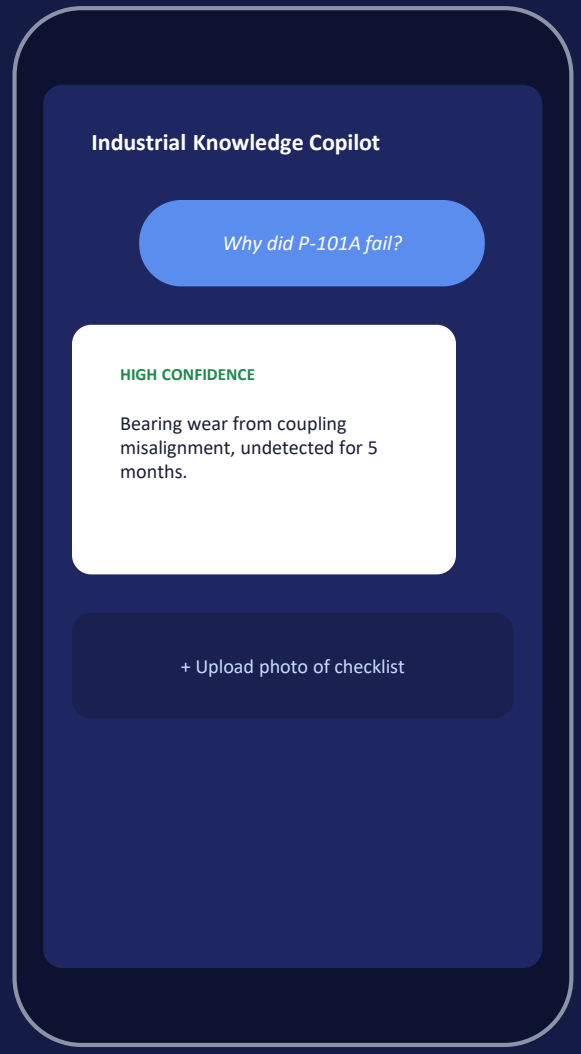
the same or a similar question gets the expert-verified answer first

Implemented in core/feedback.py — a JSON-backed store today; drop-in replacement for a real database as usage scales.

INTERFACE

Built for the field, not just the control room

- Single responsive layout — the same UI works unmodified on a phone browser
- Field technicians can photograph a paper checklist or scanned form and upload it directly — OCR ingests it on the spot
- Every answer is terse-by-default with tap-to-expand sources, designed for a small screen and a gloved hand
- Engineers at a desk get the same data with fuller source context and the entity/knowledge-graph view



ROADMAP

From hackathon prototype to production system

NOW

TF-IDF retrieval, extractive answers, offline-first

NEXT

Swap in embeddings + a vector DB (Qdrant / pgvector) for semantic recall

THEN

Promote the tag graph into a real knowledge graph (Neo4j) for multi-hop queries

FUTURE

Extend into RCA and compliance-evidence agents on the same ingestion layer

SUBMISSION

Working prototype, ready to run

Working prototype	Flask app + 5 interlinked sample documents, tested end to end
Architecture diagram	Seven-stage pipeline, this deck (slide 4)
Presentation deck	This document
Demo video	Desktop engineer query + mobile field-technician query